

FC7300_Firmware_UserManual

FC7300_Firmware_UserManual

- 1. 环境准备
- 2. 软件准备
- 3. 烧录说明
 - 3.1 修改NVR配置
 - 3.2 烧录方法一：
 - 3.2.1 bin文件转数组
 - 3.2.2 修改Id文件
 - 3.3 烧录方法二：
- 4. 验证
 - 4.1 验证固件烧录数据是否正确：
 - 4.2 验证固件功能是否正常：
 - 4.2.1 增加RSA的mailbox cmd
 - 4.2.2 RSA4096举例

Chip	Version	Release Date	Description
FC7300	v0.01	2024/07/24	Add KeyId Import
	v0.02	2024/08/05	1.Add RSA PSS Verify 2.Add ECC SIGN
	V0.03	2024/09/18	Fix Doc

1. 环境准备

- 1. Jlink9 烧录器
- 2. FC7300F8MMDT2A320T1B 芯片 即Flagchip 7300 V1.1的芯片
- 3. 《FC7300 Reference Manual_V1.1.pdf》 FC7300用户手册

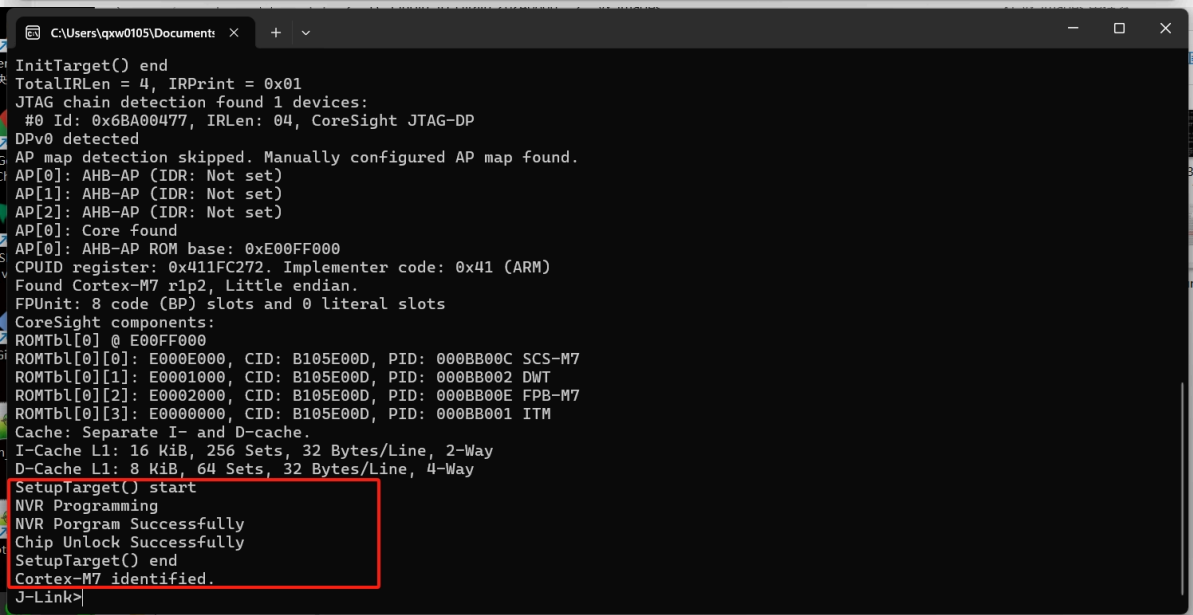
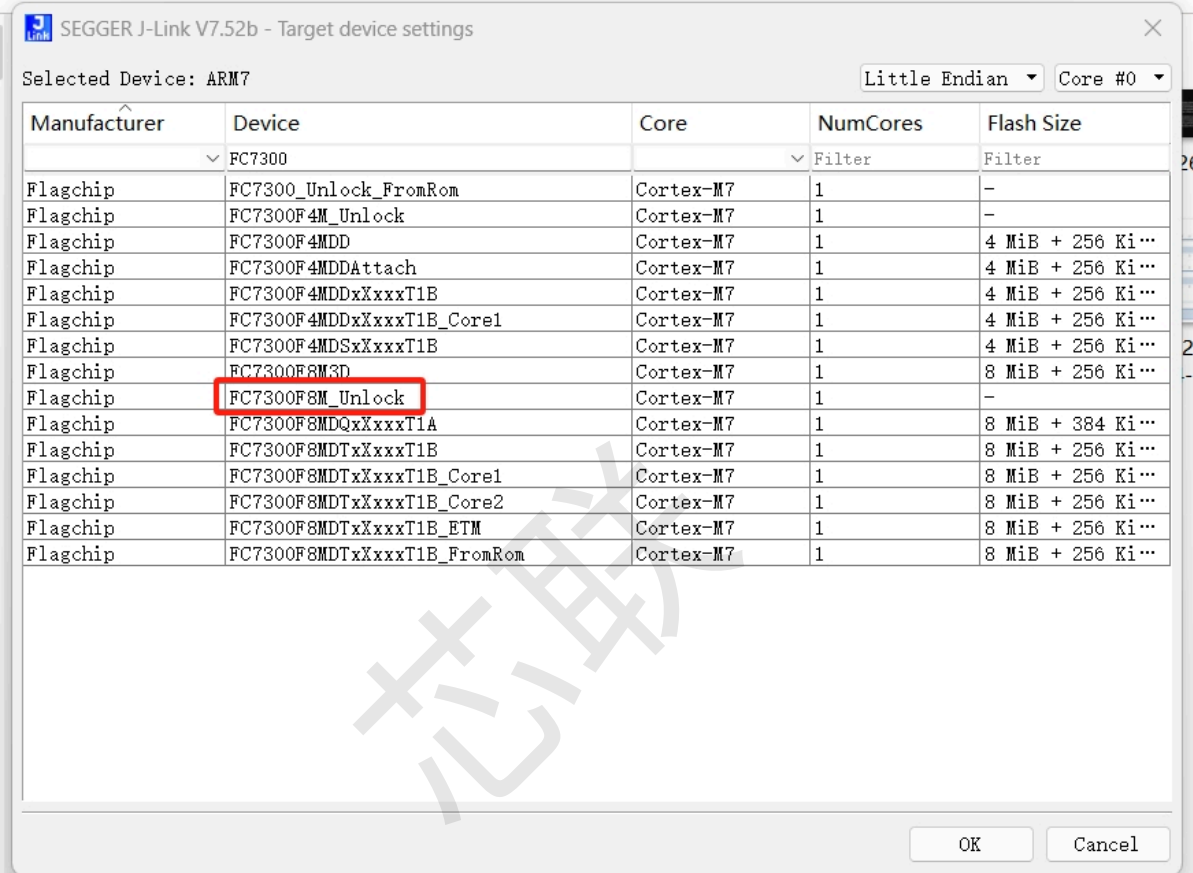
2. 软件准备

- 1. python脚本(bin文件转数组)
- 2. Flagchip 7300固件文件

3. 烧录说明

3.1 修改NVR配置

首先使用FC7300_Unlock脚本，对NVR进行解锁。这么操作的原因是需要写入NVR中的Firmware区域，并且也需要对NVR进行配置Firmware的入口地址。操作如下图。
选择FC7300F8M_Unlock设备，会显示Chip Unlock Successfully.



退出jlink终端。重新打开一个终端，再次选择设备选择正常的芯片型号即可。否则下次打开还是会使用Unlock设备，再次program NVR。

注：Unlock操作其实是把NVR数据和UserCode数据进行erase，在重新reset后，NVR的值会重新自动烧录，但是UserCode不会。因此进行Unlock操作之后，需要用户先把UserCode下载，确保芯片在正确的状态再进行下一步操作。

由于客户手中的芯片，默认是没有固件的。在加载了固件之前，需要在NVR中配置固件的入口启动地址，对应的NVR地址为0x04400084

运行Jlink.exe,设备选择 FC7300F8MDTXXXXT1B.

```
Type "connect" to establish a target connection, '?' for help
J-Link>connect
Please specify device / core. <Default>: FC7300F8MDTXXXXT1B
Type '?' for selection dialog
Device>
Please specify target interface:
  J) JTAG (Default)
  S) SWD
  T) cJTAG
TIF>
Device position in JTAG chain (IRPre,DRPre) <Default>: -1,-1 => Auto-detect
JTAGConf>
Specify target interface speed [kHz]. <Default>: 4000 kHz
Speed>
Device "FC7300F8MDTXXXXT1B" selected.

Connecting to target via JTAG
ConfigTargetSettings() start
ConfigTargetSettings() end
InitTarget() start
```

输入以下指令，

```
mem32 0x04400084 1 //读取0x04400084地址的32b的数据

w4 0x04400084 '固件入口地址' //在0x04400084地址的写入'固件入口地址'

例如: w4 0x04400084 0x11f8000
```

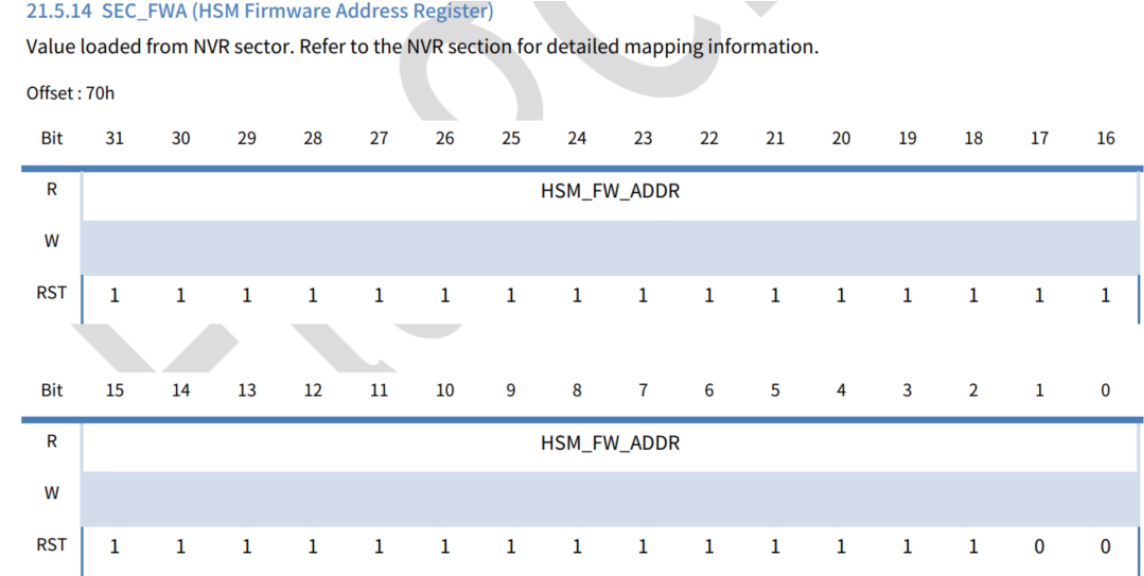
以上指令，表示固件的起始位置是在0x11f8000。

注意，对于固件的入口地址，有两个地址范围的地址可以，建议地址16字节对齐。

```
0x01000000 到 0x01000000 + (2*1024*1024) - (32*1024)，比如说 0x11f8000
0x01400000 到 0x01400000 + (2*1024*1024) - (32*1024)，比如说 0x15f8000
```

就是两个[0x01000000, 0x1f8000]，或者[0x01400000, 0x15f8000]两个地址范围内挑选一个16字节对齐的地址就行。因为NVR里数据一旦烧录，在车上就改不了了。所以固件烧录地址需要选好。

以上指令是对NVR进行配置。芯片reset以后，NVR的值会重新加载至SEC寄存器中，可以参考《FC7300 Reference Manual_V1.1.pdf》第21章。NVR中的04400084地址，对应的就是SEC寄存器中的 SEC_FWA (HSM Firmware Address Register)，地址为40014070



SEC_FWA field descriptions

Field	Type	Description
31-0 HSM_FW_ADDR	RO	HSM Firmware Address

读取0x40014070地址内容和0x04400084内容一致，则NVR配置完成。之后输入“q”指令退出终端。下一步就开始烧录固件。

```
J-Link>mem32 40014070 1
40014070 = 011F8000
J-Link>mem32 04400084 1
04400084 = 011F8000
J-Link>
```

上述0x04400084地址就是Firmware入口地址，另一个需要注意的地址为0x04400080这个地址。该地址对应SEC寄存器中的 SEC_BRC1 (BootROM Configuration Register1)，地址为4001406C。

该地址的bit10-bit8对应配置OSCFREQ，OSC的频率设置需要根据客户自身设计的板载外部晶振来进行设置，如果设置和外部晶振频率不符合，会导致部分功能异常，例如产生真随机数。在此之前优先设置bit11:OSCA为0，表示使用OSC晶振。

具体设置值可以参照下图或者《FC7300 Reference Manual_V1.1.pdf》 FC7300用户手册。

21.5.13 SEC_BRC1 (BootROM Configuration Register1)

Value loaded from NVR sector. Refer to the NVR section for detailed mapping information.

Offset : 6Ch

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	
R									SW_C								
									FG_I								
									SP_E								
									N								
W																	
RST	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	DBG_MB_BKD_EN				OSCA	OSCFREQ			CANBR		UARTBR		FW_VALID			
W																
RST	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

10-8 OSCFREQ	RO	OSC Frequency 000b - 8MHz 001b - 12MHz 010b - 16MHz 011b - 24MHz 100b - 25MHz 101b - 32MHz 110b - 48MHz 111b - 40MHz
-----------------	----	--

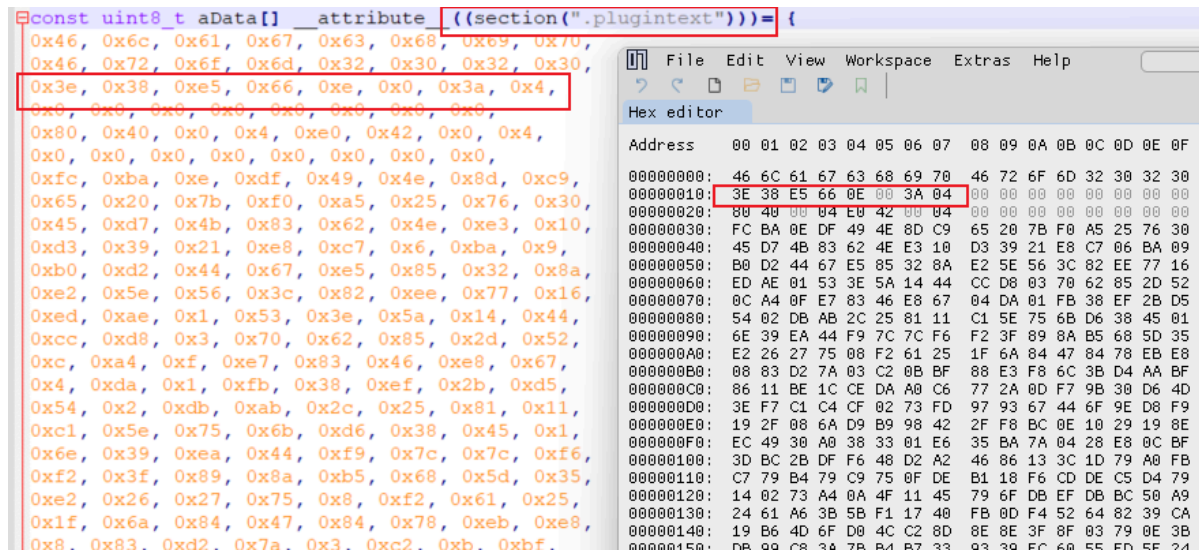
3.2 烧录方法一：

3.2.1 bin文件转数组

Flagchip对外部释放的固件文件都是bin文件。将固件文件和bin2u8array.py放在同一目录下，在当前目录打开cmd终端，运行以下指令。会在相同路径下产生 output.c 的C源码文件，内容是bin文件转为的u8类型数组。

```
python bin2u8array.py xxxxx.bin output.c
```

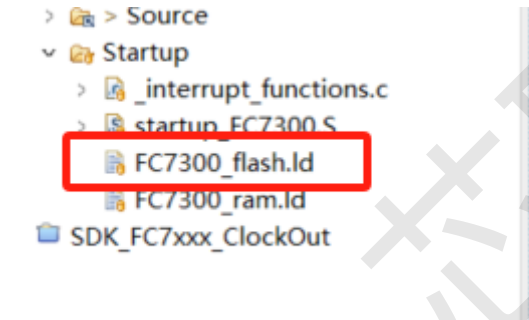
效果如图：



把生成的数组定义在在用户代码中。执行程序，对应的数据即保存到对应PFLASH中。

3.2.2 修改ld文件

为了给上述数组在Pflash区域，需要修改链接文件，为固件分配一个section来存放数组数据。



在memory中增加一个分配PLASHFW区域，起始位置为0x011f8000,size为0x8000：

```
/* Specify the memory areas */
MEMORY
{
    ITCM                    (RW) : ORIGIN = 0x00000000, LENGTH = 0x00010000

    PFLASH                  (RW) : ORIGIN = 0x01000000, LENGTH = 0x00200000 /* 4MB */
    PFLASHFW                (RW) : ORIGIN = 0x011f8000, LENGTH = 0x00008000

    /* DTCM */
    DTCM                    (RW) : ORIGIN = 0x20000000, LENGTH = 0x00020000 /* 128K */

    /* SRAM */
    SRAM_0                  (RW) : ORIGIN = 0x21000000, LENGTH = 0x00020000
    SRAM_1                  (RW) : ORIGIN = 0x21020000, LENGTH = 0x00020000
    SRAM_2                  (RW) : ORIGIN = 0x21040000, LENGTH = 0x00040000
}
```

在Section中，声明fwtext字段的数据保存在PFLASHFW区域，这个区域的数据是4字节对齐。

```

25
26/* Define output sections */
27SECTIONS
28{
29
30    /* The program code and other data goes into internal flash */
31    .text :
32    {
33        . = ALIGN(1024); /* VTOR must be align to 1K */
34        __VECTOR_TABLE = .;
35        __vector_table = .;
36        __interrupts_start__ = .;
37        . = ALIGN(4);
38        KEEP(*(.isr_vector))
39        __interrupts_end__ = .;
40        . = ALIGN(4);
41        *(EXCLUDE_FILE(*main.o *fc7xxx*.o *FPGA_Debug.o) .text)           /* .text sections (code) */
42        *(EXCLUDE_FILE(*main.o *fc7xxx*.o *FPGA_Debug.o) .text*)
43        *(EXCLUDE_FILE(*main.o *fc7xxx*.o *FPGA_Debug.o) .rodata)
44        *(EXCLUDE_FILE(*main.o *fc7xxx*.o *FPGA_Debug.o) .rodata*)
45        *(.init)                    /* section used in crt1.o files */
46        *(.fini)                    /* section used in crt1.o files */
47        *(.eh_frame)                /* section used in crtbegin.o files */
48        . = ALIGN(4);
49    } > PFLASH
50
51    .fwtext :
52    {
53        . = ALIGN(4);
54        KEEP(*(.pluginintext))
55    } > PFLASHFW
56

```

以上步骤，则完成烧录固件的方法1。只需要直接运行代码，则固件数据写入对应PFLASH区域。该方法较为繁琐，需要修改ld文件，并且如果更新固件，需要再次进行bin文件u8数组的操作。

3.3 烧录方法二：

运行Jlink.exe,设备选择 FC7300F8MDTXXXXXT1B.

直接使用Jlink的loadfile 指令加载bin到指令区域(可以直接拖拽bin文件到终端中，会自动显示文件路径)，指令如下，

```
loadfile    C:\xxxx\xxxx\xxx\xxx.bin    固件入口地址
```

```
例：loadfile    C:\Users\qxxw0105\Desktop\FC7300_firmware_test.bin    011f8000
```

```

J-Link>mem32 40014070 1
40014070 = 011F8000
J-Link>mem32 04400084 1
04400084 = 011F8000
J-Link>loadfile C:\Users\qxxw0105\Desktop\FC7300_imm_encrypt_mrk0_20240914.bin 011f8000
Downloading file [C:\Users\qxxw0105\Desktop\FC7300_imm_encrypt_mrk0_20240914.bin]...
HandleBeforeFlashProg() start
HandleBeforeFlashProg() end
J-Link: Flash download: Bank 0 @ 0x01000000: Skipped. Contents already match
O.K.
J-Link>

```

显示O.K.则为烧录成功。该方法前提是需要明确固件的入口地址。也可以使用指令mem32 011f8000 32 ,可以查看固件位置数据的正确性。


```

fc7xxx_driver_hsm.c
599 }
600
601 HSM_StatusType HSM_SSA_PSS_Verify(HSM_CmdType *pCmd, const HSMCom_RsaSsaPssVerifyType *pCfg)
602 {
603     return hsm_assign_val(pCmd, (HSM_CMD_ID_RSA_SSA_PSS_VERIFY << 16), (uint32_t)pCfg);
604 }
605

fc7xxx_driver_hsm.c
67     HSM_CMD_ID_DH_PBKDF2           = 0xe74fu,
68     HSM_CMD_ID_DH_GMPBKDF         = 0xa371u,
69     HSM_CMD_ID_SCATTER_HASH       = 0x465du,
70     HSM_CMD_ID_RAW_AESM           = 0x42fdu,
71     HSM_CMD_ID_SCATTER_MAC        = 0xa9e1u,
72     HSM_CMD_ID_RSA_SSA_PSS_VERIFY = 0x0076u,
73 } HSM_CmdIdType;
74

```

在fc7xxx_driver_hsm.h头文件中添加HSMCom_RsaSsaPssVerifyType 结构体的定义以及相关参数说明:

```

#define HSM_RSA_SSA_INPUT_RAW_MESSAGE    0u

/**
 * @brief RSA SSA input data is the hash value of raw message for
HSM_RsaSsaInputType
 *
 */
#define HSM_RSA_SSA_INPUT_HASH_DATA      1u

/**
 * @brief type definition for rsa ssa input data type
 * refer to HSM_RSA_SSA_INPUT_RAW_MESSAGE, HSM_RSA_SSA_INPUT_HASH_DATA
 */
typedef uint32_t HSM_RsaSsaInputType;

#define HSM_HASH_MD5                     (0U)
#define HSM_HASH_SHA160                   (1U)
#define HSM_HASH_SHA224                   (2U)
#define HSM_HASH_SHA256                   (3U)
#define HSM_HASH_SHA384                   (4U)
#define HSM_HASH_SHA512                   (5U)
#define HSM_HASH_SHA512_224               (6U)
#define HSM_HASH_SHA512_256               (7U)
#define HSM_HASH_SM3                      (8U)
typedef uint32_t HSM_HashAlgType;

typedef struct {
    uint32_t          u32EmBitCnt; /* RFC8017 require set it to (N's bit count
- 1). the em data bit count, it MUST < N's bit count, should keep same with the
signature generate configuration */
    HSM_HashAlgType   eMgfHashType;
    HSM_HashAlgType   eHashType; /*!< the hash type used by signature */
    uint32_t          u32SaltByteCount; /*!< the byte count of salt data, it
must > 0, suggest use hash length, for example, if eHashType is HSM_SHA_256, set
this to 256/8=32bytes */

```

```

    HSM_DataFormatType eInputDataFmt; /*!< input data(raw message or hash data)
    organized format, uint8_t array, or uint32_t array */
    HSM_RsaSsaInputType eInputType; /*!< input data type, may raw input message,
    or the hash data of raw message */
    const uint8_t      *pInputData; /*!< input data */
    uint32_t           u32InputDataByteCount; /*!< the byte count of input data,
    when the eInputType is HSM_RSA_SSA_INPUT_RAW_MESSAGE */

    HSM_DataFormatType eSignDataFmt; /*!< Signature/Public key/N input data
    organized format, uint8_t array, or uint32_t array */
    const uint8_t      *pSignData; /*!< input signature data */
    uint32_t           u32SignDataByteCount; /*!< the byte count of signature
    data, it must > 0 */

    HSM_DataFormatType ePublicKeyDataFmt;
    const uint8_t      *pPublicKey; /*!< public key, input key E data for "A =
    (input data)^E mod N" */
    uint32_t           u32PublicKeyByteCnt; /*!< the byte count of key E */

    HSM_DataFormatType eKeyNDataFmt;
    const uint8_t      *pKeyN; /*!< input key N data for "A = (input data)^E mod
    N" */
    uint32_t           u32KeyNByteCount; /*!< key N data byte count */

} HSM_RsaSsaPssVerifyType;

typedef struct {
    HSM_MailboxApiRetType u32HSMStatusRet;
    uint32_t              u32Timeout;
    uint32_t              u32UserKeyID_KEY_E;
    uint32_t              u32UserKeyID_KEY_N;
    uint32_t              u32RsaBitCnt;
    HSM_RsaSsaPssVerifyType tCfg;
}HSMCom_RsaSsaPssVerifyType;

```

以上，就可以调用HSM_StatusType HSM_SSA_PSS_Verify(HSM_CmdType *pCmd, const HSMCom_RsaSsaPssVerifyType *pCfg)接口，发送mailbox的指令，进行RSA PSS Verify算法。

4.2.2 RSA4096举例

以下例子是RSA PSS Verify 4096的例子：

```

{
    HSM_CmdType          tCmd;
    uint32_t              u32ResultLen = 0;
    uint8_t              aTestDataByteArray[0x400]; //testdata message
    uint8_t              aN[512]; //mod
    uint8_t              aSignResult[516]; //result

    uint32_t              u32BitCnt=4096; //
    uint32_t              u32NByteCnt = 0u; //mod length
    uint32_t              u32AByteCnt = 0u; //message length

```

```

uint32_t          u32EByteCnt = 0u;//public key length
uint32_t          u32InputType = 0;

HSMCom_RsaSsaPssVerifyType tVerify;
uint8_t           aPublicKey[512];

do{
    tVerify.u32Timeout                = 900000;
    tVerify.u32UserKeyID_KEY_E        = 0;
    tVerify.u32UserKeyID_KEY_N        = 0;
    tVerify.tCfg.eKeyNDataFmt          = HSM_DATA_FORMAT_1B;
    tVerify.tCfg.ePublicKeyDataFmt     = HSM_DATA_FORMAT_1B;
    tVerify.u32RsaBitCnt               = u32BitCnt;
    tVerify.tCfg.eSignDataFmt          = HSM_DATA_FORMAT_1B;
    tVerify.tCfg.eInputDataFmt         = HSM_DATA_FORMAT_1B;
    tVerify.tCfg.u32EmBitCnt           = u32BitCnt-1;
    tVerify.tCfg.eHashType             = HSM_SHA_256;
    tVerify.tCfg.eMgfHashType          = HSM_SHA_256;
    tVerify.tCfg.u32SaltByteCount      = 256/8;
    tVerify.tCfg.eInputType            = u32InputType;
    tVerify.tCfg.pInputData            = &aTestDataByteArray[0];
    tVerify.tCfg.u32InputDataByteCount = u32AByteCnt;
    tVerify.tCfg.pSignData              = &aSignResult[0];
    tVerify.tCfg.u32SignDataByteCount  = u32ResultLen;
    tVerify.tCfg.pPublicKey            = &aPublicKey[0];
    tVerify.tCfg.u32PublicKeyByteCnt   = u32EByteCnt;
    tVerify.tCfg.pKeyN                 = &(aN[0]);
    tVerify.tCfg.u32KeyNByteCount      = u32NByteCnt;

    HSM_SSA_PSS_Verify(&tCmd, &tVerify);
    MBSend(tCmd);
    if (tVerify.u32HSMStatusRet != MAILBOXAPI_RET_PASS)
    {
        break;
    }

}while(0);

asm("nop");
}

```

验证功能。根据上述例子中的RSA PSS verify的配置，进行测试。

```
n_bits:2360
InputType:1
RAW Message len:142
py_SHA_hash_data:c5d0052812c4e119044d7b675c2a66be07c7cc30ebf47e770b45acac26ce929d
Mod:cdcf3cd0d1b4939f20c2b05ef298bc070607ca2a673247b7042ca327fc772fa1c03e51f9afc45cbc86c959fd287afe3c4688bf60304509af4b929
2bde77b0934c050a593368167bf47da179324d1da10aa51da747c3aae93f9fdb749d88915dc144a0d89de60dbbecb3f5da77d4d8d81acdcd4ae1d5f
3b1f093e673f4f4b855ce7845319a59f599a777d80408bb0aa0f5328041ff81d11a34635700af3b5e3bcf68dbe9f0fc899b6c6d0f850e05bb25e6a32
d2d657a37504f80b6bebf398717282061822928de08c46934c5ba8fb54a207697db7d7d55238a313e25a8d406fbc88c9435fffbdb80595b57e26f83cc
e9d3e2d35c71c0b1895b27d13795f32a41222fb14d658e88ea4603e3db355d0ccf6e33f308cd2c4bb5e8d151fd40f14d9dacbce70b9ca0da79
Public Key :0x10001
Python Signature :16a04e44c7215adfc225a94b4eaba3bd1d3beac9885390110669f91869596e8414a9e32bf10a2915bf3608b71b9ce0721f2b1
e3bcd2e591aedcc176b4f61cf24aa94ff210dac2b279b9cd71cbfebe460f74a48677f3c0eec17c7501221d5823d75ab86b483049d9d59a21bd02ad37
6f954e925b28b93b7dc6602e7b46332cd1adf8303e225602609f86dea1381e89e524425e3343a10629e30c4bca53b5bd82ac1cc4f626efc8cb6dfadf
571143c02d55095dd50024c4e7b7d3d58c1daedca245a1d8ccbfbb41c9d7d9b0047b99e9624ee48a28d36b79fb9dc0c5b1b2fea277a9cf0a8f621281
b9442224f4b7b4c84d05627eb0e8d00e4202dc97f1242bdd5975509c9f921dab1b91b2a9ac93283ecdb4db00ed568ed816de2d6a7415ff5161b90fa
4b8aca7c7
hsm status:3
RSA SSA PSS Verify Test Success
total test counter: 1. fail counter: 0

n_bits:2896
InputType:0
RAW Message len:124
RAW Message:3c8fd9b06961756bea209ed90b7a821fbadfe1d20fee4b2e9019fd63ecd41be8687bdadb1869a019f635c070a52cf40e195a3a9469d
060a01d0c05b4a136fd44cd2be660c221d3b969c4c90b3b562d1623e7f8f3171d435da8409367f70e01d0809633c32d1b34f856d3b30be08a32f9c55
6bf16c16982fbb867853
python self test verify result: True
Mod:efc382970d669c090a8b34fac0dd4d707dd1a4b80fe3586453fcbdd2338b30cc9348939994dabf672c42fb47358146868bccf70c0f8a12bc374d8
fcd77f3c15674e1ab2c987fe287a2e67bad71dde19ea9216eeffc47f8da0ab972f3fb0701a152bec94eee84e6371521d4d2142dce358c64dc9ee7fe6
8ad59bfa61f15508e6ba0ed0d5363556873b389d04b1830c5a7647dd23d5a75017ec7a04f5833ca1029e5d6e1420c72a893b32e61b69f489a5b39ab
b346b989aff26527599357ca4c693952ee505e5750e264442259560ec26482ea6f058dc9de2b391e95c4edddad0f79cc9954491007a2e4087f9c9aa26
6696c98c68270e67da3f1b049740582c08580da50630df3350acf5750adad32935e2c718a77a3933994a29d4cd98c64de7c61e256d58bfa6ee18ab4b
9a1a60f366b24542cfb49483e9742c4449b21e1767d3699801cd05c46c4f9fa8a2bb883ed5ae7523a0ef99d57ee7a42287ed4843ebd7aa0af548fe1
2f0dea4d
Public Key :0x10001
Python Signature :2e47f44a31ee987d32116a584022e3993d6964615f527293ac9cfe8042c148479b22aa12977c3a10cb4714d0d78da8758af4
fb014eb3b58b8e48616be3542b093d6564e1338f2e64e5a3da60cb0a88e61efb811bc963ee5255808d1a4a5acc9e886ed7e9de88215863834c5263a
d7850e8666291648d6a22a0394a041ab594e95d08e2c881608787e4fff4a7d4096d7ec5b7bb5aab6b46fbb25218ae1e6887c45c554bc437caec88d03c
1dd996a042994d4f98647f07af7dcba10c3d8967e3a7f24d4234618c9bb2f8241cafc285c49184f0b46f6a607079588792c7c98abfbecaf9c8acae498
745e613bce629e7cc5bbf54364d2840f7df24168af49c57617cd84db69475b28be338e386ecf14f4e51fb66c4124e5c340ecb3cea962ad659b7b6ba
3fc0f4e22a4d8ef5bab6ada0da9aa6d9fe33ff43d10b4081ccd6de21a31aa33fe5f84f273f7dd88b39dbb94c593cb73d836e534ed56fd67456a958d
3d68f492baba9f96ab32263
hsm status:3
RSA SSA PSS Verify Test Success
total test counter: 3. fail counter: 0
```